Java w IntelliJ IDEA - tablice

BLOK 8 - tablice jedno- i dwuwymiarowe oraz obliczenia na elementach

1. Po co potrzebujemy tablic?

Zwykła zmienna przechowuje jedną wartość, np. wiek jednej osoby. Jeśli chcemy przechować wyniki 30 uczniów, tworzenie osobnych zmiennych `score1`, `score2`, `score3` i kolejnych byłoby niewygodne. Tablica pozwala umieścić wiele wartości jednego typu w jednym obiekcie. Do konkretnego elementu odwołujemy się przez indeks.

```
int[] scores = {70, 85, 90};
System.out.println(scores[0]);
System.out.println(scores[1]);
System.out.println(scores[2]);
```

```
Wynik:
70
85
90
```

Słowo `scores` oznacza zmienną wskazującą na tablicę, a zapis w nawiasach kwadratowych wybiera element. Indeks 0 oznacza pierwszy element, a nie „zerowy wynik”. Dla trzech elementów ostatnim prawidłowym indeksem jest 2. Ta zasada obowiązuje w wielu innych językach programowania.

2. Deklaracja i tworzenie tablicy

Tablicę można od razu wypełnić wartościami albo utworzyć o określonej długości i uzupełniać później. Długość zwykłej tablicy w Javie jest ustalana podczas tworzenia i później nie można jej zmienić. Zapis `new int[4]` tworzy tablicę z czterema miejscami na liczby całkowite.

```
int[] values = new int[4];
values[0] = 10;
values[1] = 20;
System.out.println(values[0]);
System.out.println(values[2]);
```

```
Wynik:
10
0
```

Elementy nowej tablicy `int[]` początkowo zawierają zera, dlatego trzeci element, którego jeszcze nie zmieniliśmy, ma wartość 0. Dla tablicy `boolean[]` domyślna wartość to `false`, a dla tablicy referencyjnej, np. `String[]`, to `null`. Nie należy utożsamiać tych wartości domyślnych z błędnymi danymi od użytkownika.

3. Pole length - ile elementów ma tablica?

Każda tablica ma pole `length` informujące o liczbie elementów. Nie jest to metoda, dlatego piszemy `numbers.length`, a nie `numbers.length()`. Dla tablicy o długości `n` dozwolone indeksy biegną od 0 do `n - 1`.

```
int[] numbers = {5, 10, 15, 20};
System.out.println(numbers.length);
System.out.println(numbers[numbers.length - 1]);
```

```
Wynik:
4
20
```

`numbers.length` zwraca 4, ale odczyt `numbers[4]` byłby nieprawidłowy. Ostatni element to `numbers[3]`. Wyrażenie `numbers.length - 1` daje właśnie ostatni dozwolony indeks i pozwala napisać kod niezależny od dokładnej długości tablicy.

Tablica	Długość	Prawidłowe indeksy
`{1}`	**1**	**0**
`{1, 2, 3}`	3	0, 1, 2
`new int[0]`	0	brak
`new int[5]`	5	0, 1, 2, 3, 4

4. Nieprawidłowy indeks - najczęstszy błąd

Jeżeli spróbujemy odczytać element spoza zakresu, Java zgłosi `ArrayIndexOutOfBoundsException`. To błąd pojawiający się podczas działania programu, a nie zwykle błąd składni wykrywany podczas kompilacji. Kompilator nie zawsze wie, jaka będzie długość tablicy i wartość indeksu w trakcie wykonania.

```
int[] data = {2, 4, 6};
// System.out.println(data[3]); // wyjątek przy uruchomieniu
```

Dobrym nawykiem jest stosowanie granicy `i < data.length` w pętli przechodzącej po całej tablicy. Warunek `i <= data.length` jest błędny, ponieważ dopuszcza indeks równy długości. Przy pustej tablicy `new int[0]` żaden indeks nie jest prawidłowy.

5. Przejście przez tablicę przy użyciu for

Pętla `for` pozwala wykonać tę samą operację dla kolejnych elementów tablicy. Licznik pętli pełni rolę indeksu, dlatego zwykle zaczyna się od 0 i zwiększa do wartości mniejszej od `length`.

```
int[] data = {4, 6, 8};
for (int i = 0; i < data.length; i++) {
    System.out.println("Indeks " + i + ": " + data[i]);
}
```

```
Wynik:
Indeks 0: 4
Indeks 1: 6
Indeks 2: 8
```

Najpierw `i = 0`, więc wyświetlana jest wartość `data[0]`, czyli 4. Przy kolejnym obrocie `i = 1` i pobieramy 6. Gdy `i` osiąga 3, warunek `3 < 3` jest fałszywy i pętla się kończy. Zaletą użycia `data.length` jest to, że kod nadal zadziała po zmianie liczby elementów.

6. Suma i średnia elementów

Aby obliczyć sumę, tworzymy akumulator z wartością 0 i dodajemy do niego każdy element. Aby obliczyć średnią, dzielimy sumę przez liczbę elementów. Jeśli chcemy wynik ułamkowy, powinniśmy zadbać o typ działania.

```
int[] points = {3, 4, 8};
int sum = 0;
for (int point : points) {
    sum += point;
}
double average = (double) sum / points.length;
System.out.println(sum);
System.out.println(average);
```

```
Wynik:
15
5.0
```

`for (int point : points)` odczytuje kolejne wartości tablicy; to wygodna odmiana pętli nazywana enhanced for. Rzutowanie `(double)` przed dzieleniem gwarantuje możliwość uzyskania ułamka. Dla pustej tablicy musimy najpierw sprawdzić `length`, aby nie próbować dzielić przez zero.

7. Znajdowanie minimum i maksimum

Najprostszy algorytm minimum zaczyna od przyjęcia pierwszego elementu jako aktualnie najmniejszego. Potem sprawdza pozostałe elementy i w razie potrzeby zapamiętuje nową, mniejszą wartość. Nie należy zaczynać od zera, bo przy tablicy zawierającej same dodatnie liczby zero nie należy do danych.

```
int[] data = {9, -2, 6, 1};
int min = data[0];
for (int i = 1; i < data.length; i++) {
    if (data[i] < min) {
        min = data[i];
    }
}
System.out.println(min);
```

```
Wynik:
-2
```

Na początku `min = 9`. Kolejny element -2 jest mniejszy, więc zmieniamy minimum. Liczby 6 i 1 nie są mniejsze od -2, dlatego wynik pozostaje bez zmian. Algorytm wymaga niepustej tablicy: bez pierwszego elementu nie można przypisać `data[0]`.

8. Modyfikowanie tablicy i zamiana elementów

Element tablicy możemy zmienić przez przypisanie, np. `numbers[1] = 100`. Aby zamienić wartości dwóch elementów, używamy zmiennej pomocniczej. Bez niej bardzo łatwo utracić jedną z pierwotnych wartości.

```
int[] numbers = {10, 20};
int temp = numbers[0];
numbers[0] = numbers[1];
numbers[1] = temp;
System.out.println(numbers[0]);
System.out.println(numbers[1]);
```

```
Wynik:
20
10
```

Najpierw `temp` zapamiętuje 10. Następnie do pierwszego elementu trafia 20. Na końcu do drugiego elementu zapisujemy starą wartość pierwszego, przechowywaną w `temp`. Ta prosta zamiana przyda się później w algorytmach sortowania.

9. Referencje do tablic - dlaczego zmieniają się dwie nazwy?

Tablica jest obiektem. Jeżeli przypiszemy jedną zmienną tablicową do drugiej, obie mogą wskazywać na ten sam obiekt. Nie jest to automatyczne kopiowanie wszystkich elementów. W efekcie modyfikacja tablicy przez jedną zmienną będzie widoczna przez drugą.

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 99;
System.out.println(a[0]);
```

```
Wynik:
99
```

Instrukcja `b = a` skopiowała referencję, a nie utworzyła drugiej niezależnej tablicy. Jeżeli potrzebujemy odrębnej tablicy o tych samych wartościach, możemy użyć `a.clone()` lub metod z klasy `Arrays`, np. `Arrays.copyOf`.

10. Tablica dwuwymiarowa jako zbiór wierszy

Tablica dwuwymiarowa to w Javie w praktyce tablica, której elementami są inne tablice. Zapis `matrix[row][column]` wybiera najpierw wiersz, a potem element w tym wierszu. Taka struktura przydaje się do macierzy, ocen kilku grup, plansz gier czy tabel.

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(matrix[0][2]);
System.out.println(matrix[1][0]);
```

```
Wynik:
3
4
```

`matrix[0]` oznacza pierwszy wiersz, a `matrix[0][2]` trzeci element pierwszego wiersza. Analogicznie `matrix[1][0]` to pierwszy element drugiego wiersza. Nie należy mylić liczby wierszy z liczbą kolumn.

11. Pętle zagnieżdżone dla macierzy

Do odwiedzenia wszystkich elementów tablicy dwuwymiarowej często używamy dwóch pętli. Zewnętrzna przechodzi po wierszach, a wewnętrzna po elementach aktualnego wiersza. `matrix.length` to liczba wierszy, a `matrix[row].length` to długość konkretnego wiersza.

```
int[][] matrix = {{1, 2}, {3, 4}};
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

```
Wynik:
1 2
3 4
```

Podczas pierwszego obrotu zewnętrznej pętli `row = 0`, więc wewnętrzna odwiedza liczby 1 i 2. Następnie przechodzimy do drugiego wiersza i wypisujemy 3 oraz 4. `println()` bez argumentu przenosi wyświetlanie do nowej linii.

12. Tablice o wierszach różnej długości

W Javie nie każdy wiersz tablicy dwuwymiarowej musi mieć tyle samo elementów. Nazywamy to tablicą poszarpaną (jagged array). Dlatego przy iteracji warto korzystać z `matrix[row].length`, a nie zakładać, że wszystkie wiersze mają długość `matrix[0].length`.

```
int[][] groups = {
    {10, 20},
    {30},
    {40, 50, 60}
};
System.out.println(groups[0].length);
System.out.println(groups[1].length);
System.out.println(groups[2].length);
```

```
Wynik:
2
1
3
```

Długości poszczególnych wierszy wynoszą 2, 1 i 3. Zapis `groups[1][1]` byłby błędny, ponieważ drugi wiersz zawiera tylko jeden element o indeksie 0. Tablice poszarpane nie są błędem; to jedna z dopuszczalnych postaci struktury danych.

Przykład 1. Example01ArrayCreate

Pokazujemy dwa sposoby przygotowania tablicy: wypełnienie wartościami i utworzenie pustych miejsc.

```
package block8;

public class Example01ArrayCreate {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40};
        String[] names = new String[3];
        names[0] = "Ala";
        names[1] = "Ola";
        names[2] = "Jan";

        System.out.println(numbers[0]);
        System.out.println(numbers.length);
        System.out.println(names[2]);
    }
}
```

Wynik:

```
10
4
Jan
```

- Instrukcja `int[] numbers = {10, 20, 30, 40};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.
- Instrukcja `String[] names = new String[3];` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.
- Instrukcja `System.out.println(numbers[0]);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(numbers.length);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

- Instrukcja `System.out.println(names[2]);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 2. Example02ArrayOperations

Przechodzimy przez elementy tablicy i wykonujemy na nich kolejne działania.

```
package block8;

import java.util.Arrays;

public class Example02ArrayOperations {
    public static void main(String[] args) {
        int[] numbers = {7, 2, 9, 4, 1};
        int sum = 0;
        int min = numbers[0];
        int max = numbers[0];

        for (int i = 0; i < numbers.length; i++) {
            int number = numbers[i];
            sum += number;
            if (number < min) min = number;
            if (number > max) max = number;
        }

        int[] copy = Arrays.copyOf(numbers, numbers.length);
        Arrays.sort(copy);

        System.out.println("Suma: " + sum);
        System.out.println("Min: " + min + ", max: " + max);
        System.out.println("Oryginał: " + Arrays.toString(numbers));
        System.out.println("Kopia posortowana: " + Arrays.toString(copy));
    }
}
```

Wynik:

```
Suma: 23
Min: 1, max: 9
Oryginał: [7, 2, 9, 4, 1]
Kopia posortowana: [1, 2, 4, 7, 9]
```

- Instrukcja `int[] numbers = {7, 2, 9, 4, 1};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.
- Zapis `int sum = 0;` tworzy zmienną `sum` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `int min = numbers[0];` tworzy zmienną `min` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Zapis `int max = numbers[0];` tworzy zmienną `max` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Nagłówek `for (int i = 0; i < numbers.length; i++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Zapis `int number = numbers[i];` tworzy zmienną `number` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.

Przykład 3. Example03TwoDimensional

Odczytujemy elementy tablicy dwuwymiarowej za pomocą indeksów wiersza i kolumny.

```
package block8;

public class Example03TwoDimensional {
    public static void main(String[] args) {
        int[][] matrix = {
                {1, 2, 3},
                {4, 5, 6},
                {7, 8, 9}
        };

        System.out.println(matrix[1][2]); // 6
        matrix[0][0] = 100;
        System.out.println(matrix[0][0]);
    }
}
```

Wynik:

```
6
100
```

- Instrukcja `System.out.println(matrix[1][2]); // 6` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(matrix[0][0]);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 4. Example04MatrixTraversal

Dwie pętle odwiedzają każdy element macierzy dokładnie w kolejności wyznaczonej przez liczniki.

```
package block8;

public class Example04MatrixTraversal {
    public static void main(String[] args) {
        int[][] matrix = {
                {2, 4, 6},
                {1, 3, 5}
        };

        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix[row].length; col++) {
                System.out.print(matrix[row][col] + " ");
            }
            System.out.println();
        }
    }
}
```

Wynik:

```
2 4 6
1 3 5
```

- Nagłówek `for (int row = 0; row < matrix.length; row++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Nagłówek `for (int col = 0; col < matrix[row].length; col++) {` określa wartość startową, warunek kontynuowania oraz sposób zmiany licznika. Trzeba sprawdzić, czy ostatnia iteracja mieści się w granicach.
- Instrukcja `System.out.print(matrix[row][col] + " ");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println();` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Zadanie 5. ArrayStats

Dla tablicy numbers oblicz sumę, średnią, minimum i maksimum. Wypisz wszystkie cztery wyniki bez sortowania danych.

```
package block8;

public class ArrayStats {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 8, 1, 9, 6};
        // TODO
    }
}
```

- Instrukcja `int[] numbers = {5, 2, 8, 1, 9, 6};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.

Zadanie 6. RowSums

Dla każdego wiersza matrix oblicz osobną sumę i ją wypisz. Użyj dwóch pętli i odwołuj się do matrix[row].length.

```
package block8;

public class RowSums {
    public static void main(String[] args) {
        int[][] matrix = {{1, 2}, {10, 20}, {3, 4}};
        // TODO
    }
}
```

Zadanie 7. ReverseArray

Odwróć tablicę numbers w miejscu, bez tworzenia drugiej tablicy. Zamieniaj elementy parami od końców i wypisz wynik.

```
package block8;

public class ReverseArray {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 6};
        // TODO
    }
}
```

- Instrukcja `int[] numbers = {1, 2, 3, 4, 5, 6};` deklaruje tablicę. Po utworzeniu odwołujemy się do jej elementów przez indeksy zaczynające się od zera.

Zadanie 8. MatrixSum

Oblicz sumę wszystkich elementów macierzy matrix za pomocą dwóch pętli i wyświetl wynik. Oczekiwana suma to 21.

```
package block8;

public class MatrixSum {
    public static void main(String[] args) {
        int[][] matrix = {{1, 2, 3}, {4, 5, 6}};
        // TODO
    }
}
```

13. Zadania - tablice jedno- i dwuwymiarowe

Zadanie 9. Utwórz tablicę `{4, 8, 12, 16}`. Wypisz liczbę elementów, pierwszy i ostatni element. Następnie zmień drugi element na 100 i wyświetl wszystkie wartości.

Zadanie 10. Dla tablicy `{5, 7, 9}` oblicz sumę i średnią. Najpierw przewidź wartości akumulatora po każdej iteracji.

Zadanie 11. Znajdź najmniejszą i największą liczbę w `{12, -4, 0, 7, -9}`. Nie korzystaj z sortowania. Pomyśl, dlaczego początkowym minimum powinien być pierwszy element.

Zadanie 12. Utwórz tablicę 2D z trzema wierszami i dwiema kolumnami. Policz sumy osobno dla każdego wiersza oraz sumę całej tablicy.

Zadanie 13. Przygotuj dwie zmienne wskazujące na tę samą tablicę, a następnie zmień wartość przez jedną z nich. Porównaj wynik z sytuacją, gdy drugą tablicę utworzysz przez `clone()`.